

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 1
Реализация REST API на основе boilerplate

Выполнила:

Персук Виктория

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

1. Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и [мой](#))
2. С учётом имеющихся данных по результатам ДЗ1 и ДЗ2 реализуйте:
 - модели
 - представления
 - контроллеры
3. В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Ход работы

Работаю над сервисом бронирования столиков в ресторане. Использовала boilerplate, который предлагает преподаватель. Модели базы данных и эндпоинты для API уже спроектированы в ходе выполнения ДЗ1 и ДЗ2.

Приложение построено по трёхслойной архитектуре:

1. config/ – подключение к БД, переменные окружения
2. models/ – typeORM-сущности
3. controllers/ – обработчики маршрутов
4. middleware/ – JWT-аутентификация
5. common/ – базовые классы, перечисления
6. utils/ – хэширование и проверка паролей

Реализованный функционал

1. Управление пользователями и аутентификация

Реализованы эндпоинты:

- POST /auth/register – регистрация по email/паролю, возвращает JWT-токен
- POST /auth/login – вход, возвращает JWT-токен
- GET /users/me – профиль текущего пользователя
- PATCH /users/me – обновление профиля
- GET /users/me/reservations – история бронирований

Пароль хэшируется через bcryptjs при сохранении с помощью TypeORM-подписчика (UserSubscriber):

```
1. async beforeInsert(event: InsertEvent<User>) {
2.   if (event.entity.password_hash && !event.entity.password_hash.startsWith('$2b$')) {
3.     event.entity.password_hash = hashPassword(event.entity.password_hash);
4.   }
5. }
```

Поле password_hash скрыто из всех ответов API с помощью декоратора @Exclude() из class-transformer (опция classTransformer: true включена глобально):

```
1. @Exclude()
2. @Column({ type: 'varchar', length: 300, nullable: false })
3. password_hash!: string;
```

Ролевая модель включает роли: Admin, User, Owner, Manager. При создании ресторана пользователь автоматически получает роль Owner.

2. Управление ресторанами

- GET /restaurants – список верифицированных ресторанов с фильтрацией по городу, ценовой категории, типу кухни
- POST /restaurants – создание ресторана (авторизованный пользователь)
- PATCH /restaurants/:id – обновление (owner / admin)
- DELETE /restaurants/:id – удаление (admin / owner)
- PATCH /restaurants/:id/verify / reject – модерация (admin)

Фильтрация реализована через QueryBuilder:

```
1. const qb = this.repository
2.   .createQueryBuilder('restaurant')
3.   .where('restaurant.status = :status', { status: RestaurantStatus.Verified });
4.
5. if (city) qb.andWhere('restaurant.city = :city', { city });
6. if (price) qb.andWhere('restaurant.price = :price', { price });
7. if (cuisineId) {
8.   qb.innerJoin('restaurant_cuisines', 'rc',
9.     'rc.restaurant_id = restaurant.restaurant_id AND rc.cuisine_id = :cuisineId',
10.    { cuisineId });
11. }
```

3. Бронирование столиков

- POST /reservations – создание бронирования с проверкой вместимости и конфликтов по времени
- GET /reservations/:id – просмотр (только своё бронирование)
- PATCH /reservations/:id – изменение (не позднее чем за 3 часа)
- DELETE /reservations/:id – отмена (не позднее чем за 3 часа)

Проверка конфликтов по времени:

```
1. const conflicting = await this.repository
2.   .createQueryBuilder('reservation')
3.   .where('reservation.table_id = :tableId', { tableId: body.table_id })
4.   .andWhere('reservation.status IN (...statuses)', {
5.     statuses: [ReservationStatus.Confirmed, ReservationStatus.Pending],
6.   })
7.   .andWhere(
8.     'reservation.reservation_time > :start AND reservation.reservation_time < :end',
9.     { start: twoHoursBefore, end: twoHoursAfter }
10.  )
11.  .getOne();
```

4. Дополнительные модули

- Меню: GET /restaurants/:id/menu, POST /restaurants/:id/menus
- Отзывы: GET/POST /restaurants/:id/reviews, PATCH/DELETE /reviews/:id
- Персонал: назначение менеджеров владельцем ресторана
- Столики: CRUD для столиков ресторана
- Фото: добавление и просмотр фотографий ресторана

Для заполнения базы тестовыми данными создан скрипт src/seed.ts. Скрипт идиempотентен (повторный запуск не создаёт дублей) и заполняет:

- 4 роли (Admin, User, Owner, Manager)
- 5 пользователей (пароль: password123)
- 5 кухонь (Italian, Japanese, French, Mexican, Georgian)
- 3 ресторана (2 верифицированных, 1 в статусе Pending)
- 5 столиков, 3 бронирования, 3 отзыва

Вывод

В ходе лабораторной работы разработан полноценный REST API для системы бронирования столиков в ресторанах. Реализована ролевая модель с четырьмя ролями, JWT-аутентификация, фильтрация и поиск ресторанов, бизнес-логика бронирования с проверкой временных конфликтов и ограничений по времени отмены. Все пароли хранятся в хэшированном виде, поле password_hash исключено из всех API-ответов.